

Homework 2 Solution

CS 355 Database Systems

Habib University

Fall 2025

Question 1: Relational Algebra

Q1:

Step 1: All (s_id, c_id) pairs where the student's department matches the course's department

$$R_1 = \pi_{s_id, c_id}(Students \bowtie_{s_dept=c_dept} Courses)$$

Step 2: All (s_id, c_id) pairs where the student has enrolled

$$R_2 = \pi_{s_id, c_id}(Enrollments)$$

Step 3: Students missing at least one course from their department

$$R_3 = R_1 - R_2$$

Step 4: Students who have not missed any such course

$$R_4 = \pi_{s_id}(Students) - \pi_{s_id}(R_3)$$

Final Answer:

$$\pi_{s_name}(Students \bowtie_{Students.s_id=R_4.s_id} R_4)$$

Q2:

$$Q_{better} = \pi_{sid}(\sigma_{grade > pgrade}(Enroll \bowtie Prereq \bowtie \rho_{pid,psid,pgrade}(\pi_{cid,sid,grade}(Enroll))))$$

$$Q = \pi_{name}(Student \bowtie Q_{better})$$

OR

Step 1: Join prerequisites with enrollments (course side)

$$R_1 = \rho_{E1}(Enrollments) \bowtie_{E1.c_id=Prerequisites.c_id} Prerequisites$$

Step 2: Join prerequisites with enrollments (prerequisite side)

$$R_2 = \rho_{E2}(Enrollments) \bowtie_{E2.c_id=Prerequisites.p_id} Prerequisites$$

Step 3: Join R_1 and R_2 on same student id

$$R_3 = R_1 \bowtie_{E1.s_id=E2.s_id} R_2$$

Step 4: Select students with higher grade in the course than in prerequisite

$$R_4 = \sigma_{E1.e_grade\text{point}>E2.e_grade\text{point}}(R_3)$$

Final Answer:

$$\pi_{E1.s_id,s_name}(R_4 \bowtie_{E1.s_id=Students.s_id} Students)$$

Q3:

Step 1: Join enrollments and courses

$$R_1 = Enrollments \bowtie_{Enrollments.c_id=Courses.c_id} Courses$$

Step 2: Select failed ones (grade\text{point} < 2.0)

$$R_2 = \sigma_{e_grade\text{point}<2.0}(R_1)$$

Final Answer:

$$\pi_{s_id,c_id,c_credits}(R_2)$$

Q4:

Step 1: Self-join prerequisites on same course but different prerequisite

$$R_1 = \rho_{P1}(Prerequisites) \bowtie_{P1.c_id=P2.c_id \wedge P1.p_id \neq P2.p_id} \rho_{P2}(Prerequisites)$$

Step 2: Get course ids from R_1

$$R_2 = \pi_{P1.c_id}(R_1)$$

Final Answer:

$$\pi_{c_title}(R_2 \bowtie_{c_id} Courses)$$

Q5:

Step 1: Courses taken by student 001

$$A = \pi_{c_id}(\sigma_{s_id='001'}(Enrollments))$$

Step 2: Courses taken by student 002

$$B = \pi_{c_id}(\sigma_{s_id='002'}(Enrollments))$$

Step 3: Courses taken by both

$$C = A \cap B$$

Final Answer:

$$(A \cup B) - C$$

Question 2: Cascadeless/Conflict Serializable Schedules

A.

T1	T2	T3
	Read(A)	
	Read(B)	
	A=B+1	
Read(A)		
Read(S)		
Read(T)		
		Read(T)
		Read(A)
		T=A+T
		Write(T)
		Commit
	Write(A)	
A=S+T		
	Commit	
Write(A)		
Commit		

B.

T1	T2	T3
	Read(A)	
	Read(B)	
	A=B+1	
		Read(T)
		Read(A)
		T=A+T
		Write(T)
	Write(A)	
Read(A)		
Read(S)		
Read(T)		
A=S+T		
	Commit	
Write(A)		
Commit		
		Commit

C.

T1	T2	T3
	Read(A)	
	Read(B)	
	A=B+1	
		Read(T)
		Read(A)

		T=A+T
		Write(T)
		Commit
	Write(A)	
Read(A)		
Read(S)		
Read(T)		
A=S+T		
	Commit	
Write(A)		
Commit		

D.

T1	T2	T3
	Read(A)	
	Read(B)	
	A=B+1	
		Read(T)
		Read(A)
		T=A+T
		Write(T)
		Commit
	Write(A)	
	Commit	
Read(A)		
Read(S)		
Read(T)		
A=S+T		
Write(A)		
Commit		

E.

No such schedule is possible because every cascadeless schedule is also recoverable.

Question 3: Precedence Graph

Lock-X(C)	Write(C)	Unlock-S(A)
Write(C)	Unlock-X(B)	Unlock-X(C)
Write(B)	Unlock-X(C)	Commit
Read(C)	Commit	
Unlock-X(A)		
Unlock-X(B)		
Unlock-X(C)		
Commit		

b)

T1	T2	T3
		Lock-S(A)
		Read(A)
	Lock-X(B)	
	Read(B)	
		Lock-X(C)
		Write(C)
		Read(A)
		Unlock-S(A)
		Unlock-X(C)
		Commit
Lock-X(A)		
Read(A)		
	Lock-X(C)	
	Read(C)	
	Write(B)	
	Write(C)	
	Unlock-X(B)	
	Unlock-X(C)	
	Commit	
Lock-X(B)		
Read(B)		
Write(A)		
Lock-X(C)		
Write(C)		
Write(B)		
Read(C)		
Unlock-X(A)		
Unlock-X(B)		
Unlock-X(C)		
Commit		

1. Write a query to return the titles and authors of all the books in the database
 - `db.library.find({}, {"title": true, "author": true});`
2. Write a query that returns all books that were published between 1900 and 1999
 - `db.library.find({$and : [{"publicationYear" : {$gte:1900}}, {"publicationYear" : {$lt:2000}}]});`
3. Write a query that returns all the books that are not checked out by any member
 - `db.library.find({"checkedOutBy" : {$exists:false}});`
4. Write a query to display all titles and authors of books having Fantasy genre
 - `db.library.find({"genre":"Fantasy"}, {"title":true, "author" :true});`
5. Write a query to get the total number of books of the Horror genre
 - `db.library.find({"genre":"Horror"}).count();`
6. Write a query to display all the books that have more than one authors
 - `db.library.find({"author.1":{$exists:true}});`
7. Write a query to insert a book with the following details:

Title = "The Mortal Instruments – City of Bones"
Authors = Cassandra Clare, Joshua Lewis
Genre = Fantasy
Published in year = 2007
Number of copies added in the library = 5
 - `db.library.insertOne({
 "title": "The Mortal Instruments - City of Bones",
 "author": ["Cassandra Clare", "Joshua Lewis"],
 "genre": "Fantasy",
 "publicationYear": 2007,
 "copiesAvailable": 5});`
8. Write a query that increments the number of copies available of all the Fiction books by 3
 - `db.library.updateMany({"genre":"Fiction"}, {$inc:{"copiesAvailable":3}});`
9. Write a query that deletes all books that are not checked out by anyone
 - `db.library.deleteMany({"checkedOutBy":{$exists:false}});`
10. Write a query that deletes the first non-fiction book in the list
 - `db.library.deleteOne({"genre":{$ne:"Fiction"}});`